

Optimized Deep Learning - Jean Zay

Training and large batches





This section focuses on a few of the training hyperparameters: the learning rate and the optimiser, as well as the optimiser's parameters.

We will look at their purpose, their effects on training, and finally how to configure them for training on large batches.

Mise en contexte : optim matérielle jusqu'ici, maintenant optim algorithmique (en particulier large batch pour DDP)

Loss Landscape

Loss Landscape ◂

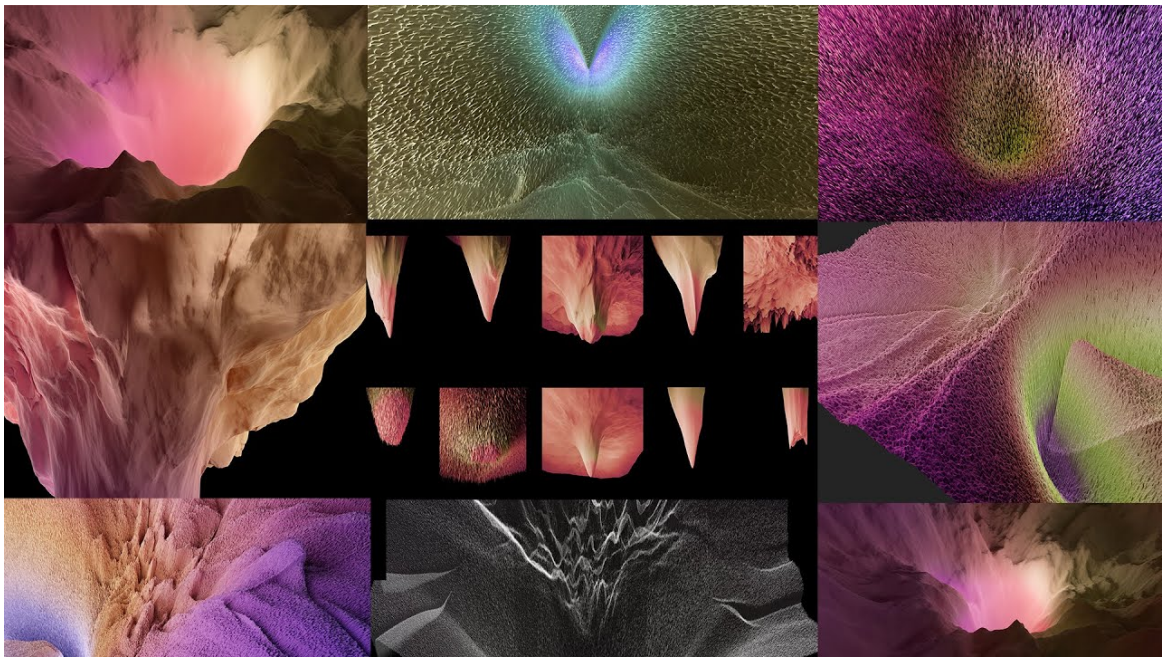
Residual Learning ◂

Initialization ◂

To begin with, let’s talk about the loss landscape. This is a tool which, although costly to set up, allows us to visualise the gradient descent and the impact of our hyperparameters on it.

Loss Landscape

https://losslandscape.com/



The images shown here are taken from the website https://losslandscape.com/, which presents numerous examples of loss visualisation and highlights various research papers on loss visualisation methods and the benefits these visualisations can offer.

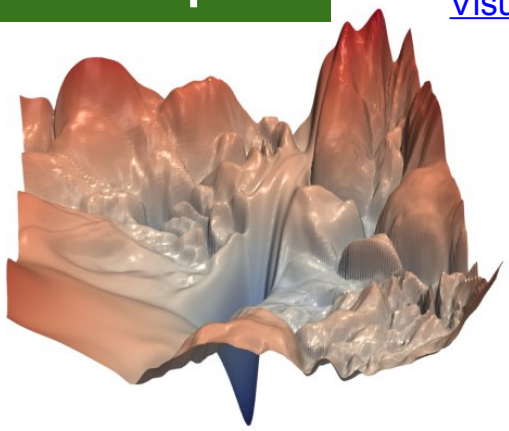
What is known as a ‘loss landscape’ is a visual representation of how the loss changes as a function of the model’s parameters. Of course, as there are often tens of millions of such parameters (even for a small model), the landscape is projected here onto a 2D representation of the parameters, with the loss as the third dimension.

Using their tool, it is also possible to visualise gradient descent: https://losslandscape.com/explorer.

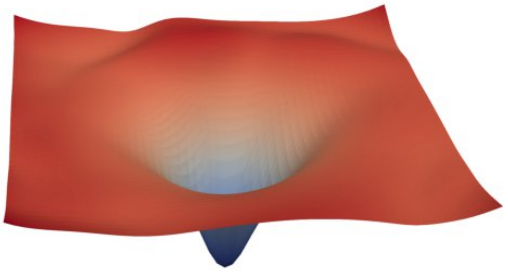
Loss Landscape

[Visualizing the Loss Landscape of Neural Nets]

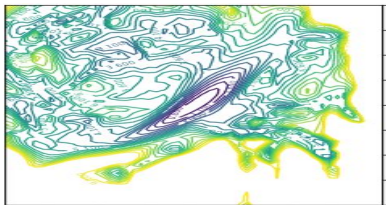
Residual Learning
Since Resnets (2015) ...

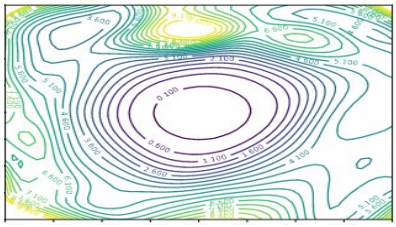


(a) without skip connections



(b) with skip connections

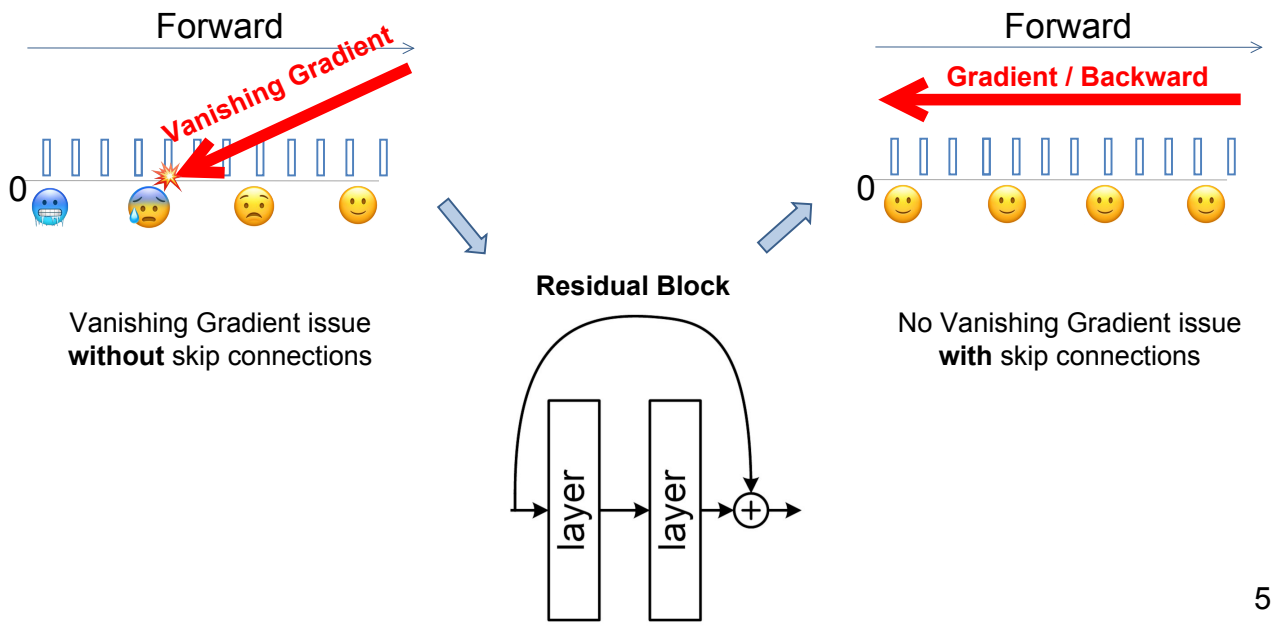




Visualising the loss helps us understand the effect of certain optimisation techniques.

For example, when comparing the visualisation of the loss during model training with and without *skip connections*, we can see that adding skip connections makes the loss landscape much closer to a convex function (a function with only one global minimum – if a local minimum exists, then it is the global minimum), which greatly facilitates gradient descent by making it less chaotic.

Residual Learning



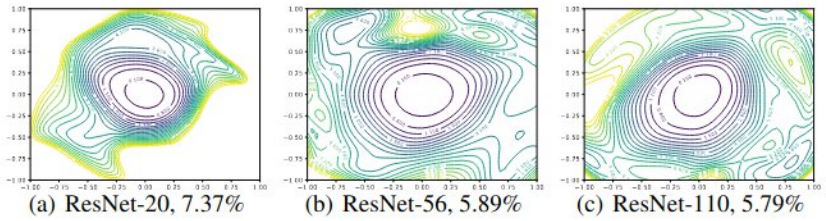
Lorsqu'un modèle commence à avoir trop de couches, on remarque que, avec le problème de *vanishing gradient* lors de la rétro-propagation, les premières couches auront un gradient nul et donc aucune modification des poids.

Pour faire face à ce problème on utilisera le *residual learning.* Ce qui correspond à faire des liens (*skip connections*) qui vont sauter une ou plusieurs couches du réseau pour donner en entrée d'une couche éloignée leur sortie qui sera additionnée à la sortie de la couche précédente. Cela permet une diffusion du gradient sur l'ensemble des couches du modèle.

Le modèle *Resnet* a introduit cette technique qui est maintenant utilisée dans tous les modèles.

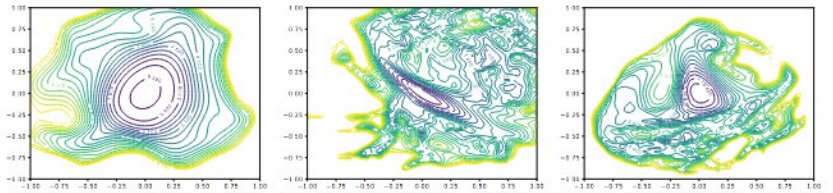
Residual Learning – depth impact

ResNet-20



(a) ResNet-20, 7.37% (b) ResNet-56, 5.89% (c) ResNet-110, 5.79%

ResNet-20
without skip connections



(d) ResNet-20-NS, 8.18% (e) ResNet-56-NS, 13.31% (f) ResNet-110-NS, 16.44%

Skip connections prevent increasing depth from making the loss landscape excessively sharp and chaotic

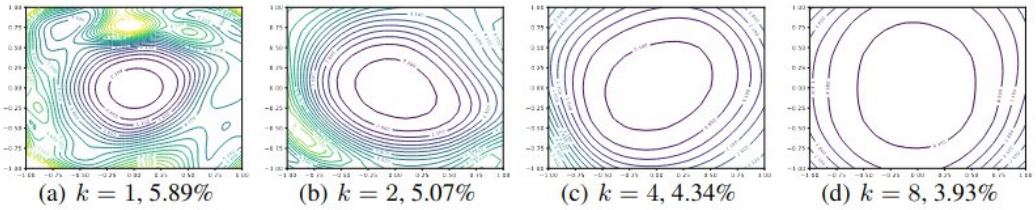
When using the ResNet model with skip connections, we can see that as the model size increases, the loss landscape becomes increasingly similar to a simple convex function.

Conversely, when using the same model without skip connections, the larger the model becomes, the more the loss takes on a form that makes gradient descent difficult.

The percentage shown corresponds to the loss at the end of training. The smaller the value, the better the training.

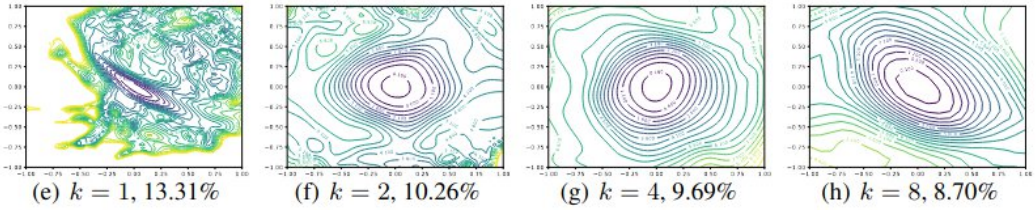
Residual Learning – width impact

Wide-ResNet-56



(a) $k = 1, 5.89\%$ (b) $k = 2, 5.07\%$ (c) $k = 4, 4.34\%$ (d) $k = 8, 3.93\%$

Wide-ResNet-56
without skip connections



(e) $k = 1, 13.31\%$ (f) $k = 2, 10.26\%$ (g) $k = 4, 9.69\%$ (h) $k = 8, 8.70\%$

Skip connections prevent narrow networks from developing highly irregular loss landscapes

k = filter coefficient compared to ResNet-56

However, in both cases (with or without skip connections), it can be observed that increasing the number of convolution filters per layer (the channel width coefficient) increases the convexity of the loss function, which facilitates learning.

Note: This explains why deep VGGs, but mainly very wide ones, worked well before Resnets and their skip connections.

The percentage shown corresponds to the loss at the end of training. The smaller the value, the better the training.

Model Parameters Initialization

The Blessing of Dimensionality :



By default in PyTorch:

- Best initialization algorithm depending on the type of layer (linear, convolutional, transform, ...).
- Today, it is no longer necessary to try to optimize initialization.

In contrast to the principle of the 'Curse of dimensionality' – which states that the more dimensions are added, the more difficult (or even impossible) learning becomes – in deep learning we observe a beneficial effect known as the 'Blessing of dimensionality' when a large number of dimensions is reached.

As the number of dimensions increases, so does the number of acceptable local minima. **The gradient descent problem therefore becomes a local problem in which we seek the acceptable local minimum closest to our starting point.**

The choice of starting point corresponds to the initialisation of our neural network. There are various algorithms that attempt to optimise this initialisation (Xavier(1), Kaiming(2), ...).

In PyTorch, initialisation is optimised by default using these different algorithms depending on the types of layers in the network. For example, according to the documentation, Kaiming sampling will be used for linear and convolutional layers, Xavier uniform for Transformers, and so on.

(1) Uniform sampling

(2) Gaussian sampling

Learning Rate Scheduler

Learning Rate scheduler ◂

Cyclic scheduler ◂

One Cycle scheduler ◂

ScheduleFree ◂

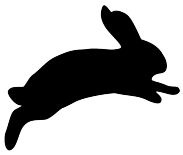
LR Finder◂

In this section, we will discuss the learning rate and methods for scheduling it.

Learning Rate

$$\underbrace{\theta}_{\text{Gradient}} = \theta - \underbrace{\eta}_{\text{LR}} \underbrace{\frac{\partial J(\theta)}{\partial \theta}}_{\text{Gradient}}$$

High Learning-rate



+ Larger steps (faster convergence)
+ Escape local minima
+ Good for early steps
- Risk skipping acceptable minima
- Risk “bouncing” over minima

Small Learning-rate



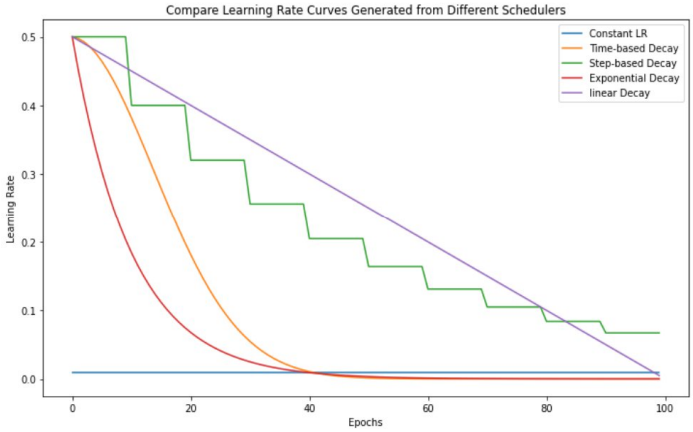
+ Don’t skip local minima
+ Converge towards bottom of minima
+ Good for later steps
- Smaller steps (slower convergence)
- Risk getting stuck in local minima

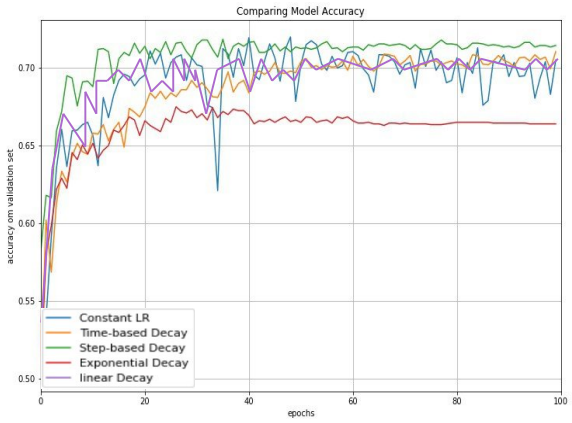
The loss landscape (https://losslandscape.com/explorer) clearly illustrates that an initially high learning rate saves time by rapidly approaching the solution, whereas a lower learning rate allows for smaller steps and a more precise approach to the local minimum.

However, a learning rate that is too low at the start will result in very slow convergence, whilst a learning rate that is too high at the end risks taking us further away from the minimum.

Learning Rate Scheduler

Learning rate decay





Fortunately, the learning rate does not have to remain constant throughout training. And based on what has just been said, we can choose to gradually decrease the learning rate during training to suit our needs.

There are several ways to do this:

Constant (blue) ==> Good performance but unstable throughout

Linear Decay (purple): The learning rate is reduced steadily with each epoch ==> Good performance, stable towards the end

Step Decay (green): The learning rate is reduced by a fixed factor every x epochs ==> Quite similar to linear decay (in terms of profile and performance. Here, it converges more quickly to the optimum because it remains high for longer)

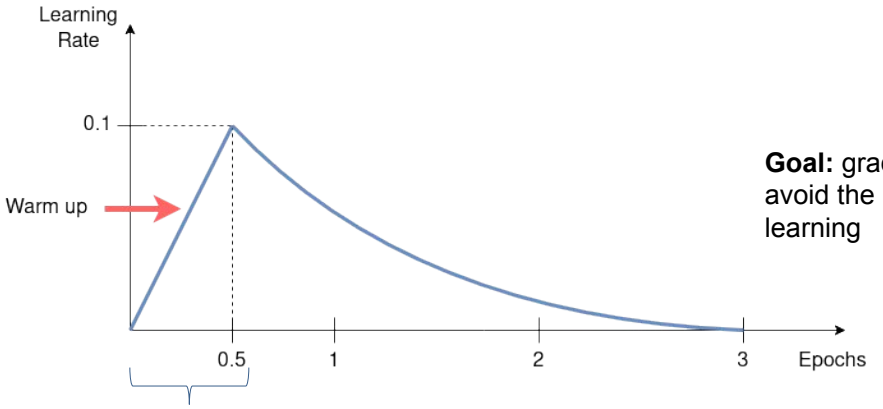
Time-based (orange): The learning rate is reduced by a factor inversely proportional to the number of epochs elapsed ==> The learning rate decreases rapidly, so it reaches the optimum more slowly than the others, but not too slowly to achieve the same performance

Exponential (red): The learning rate is reduced exponentially (quite simply)

Learning Rate Scheduler

WARMUP for large batches

Problems: The first iterations have too much effect on the model (significant losses, high gradients, bias, etc.), a high learning rate can cause strong instability or divergence



Untrusted Gradients

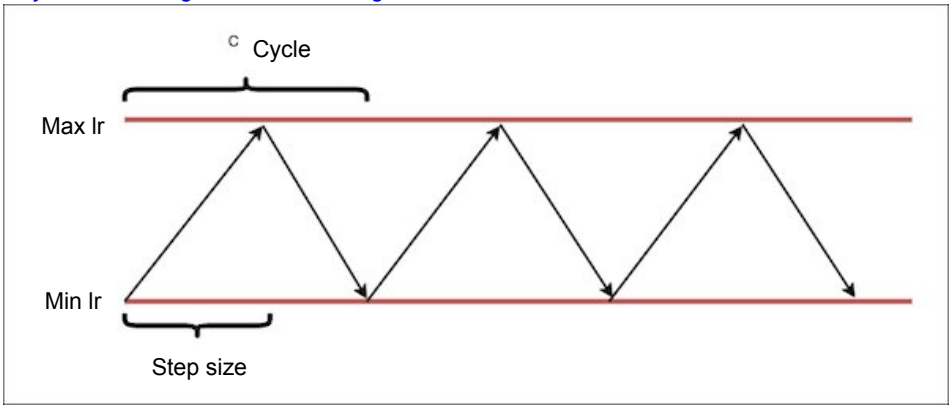
Goal: gradually increase the learning rate to avoid the risk of divergence at the start of learning

Despite techniques to optimise the random initialisation of weights, the initial iterations can still have too great an effect on the model's updates (high loss, steep gradients, bias, etc.). This effect is all the more pronounced with large batch sizes.

To try to reduce the effects of these initial iterations, a warm-up phase (https://arxiv.org/pdf/2406.09405) can be added alongside the learning rate scheduler; this starts with a low learning rate and gradually increases it at the beginning of training.

Cyclic Learning Rate Scheduler

Cyclical Learning Rates for Training Neural Networks - Leslie N. Smith 2017



Paramètres :

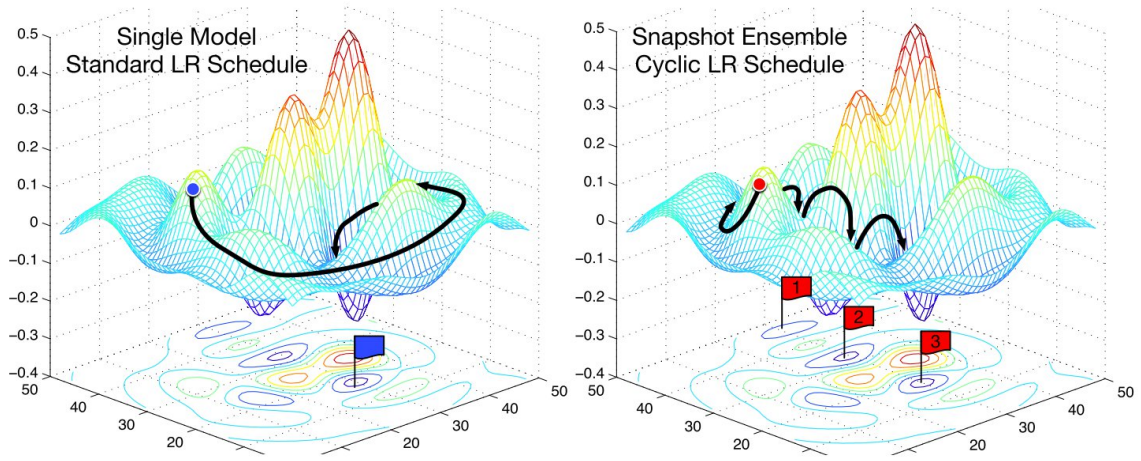
- Step_size = x * epoch (2 ≤ x ≤ 10)
- Base_lr -> min convergence value
- max_lr -> max convergence value

Succession of warmups and learning rate decays

Building on the idea of a learning rate that is dynamically adjusted during training, Leslie N. Smith introduced the concept of the Cyclic Learning Rate Scheduler in 2017.

The principle involves alternating between warm-up phases and phases where the learning rate is reduced. The learning rate varies between a minimum and a maximum value and repeats these cycles, the duration of which is proportional to the number of iterations per epoch.

Cyclic Learning Rate Scheduler



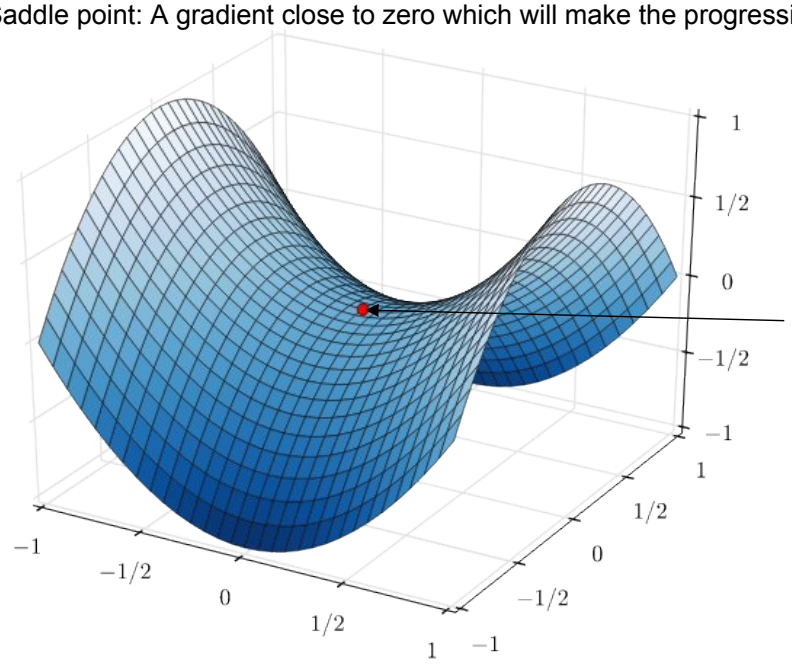
SNAPSHOT ENSEMBLES: TRAIN 1, GET M FOR FREE
Gao Huang, Yixuan Li, Geoff Pleiss

14

The main advantage of this technique is that it forces the model to easily escape from potential local minima in order to try and find a better one each time the learning rate increases. It also enables the model to escape from ‘saddle’ points with low gradients.

Saddle point Problem

Saddle point: A gradient close to zero which will make the progression of the model very slow



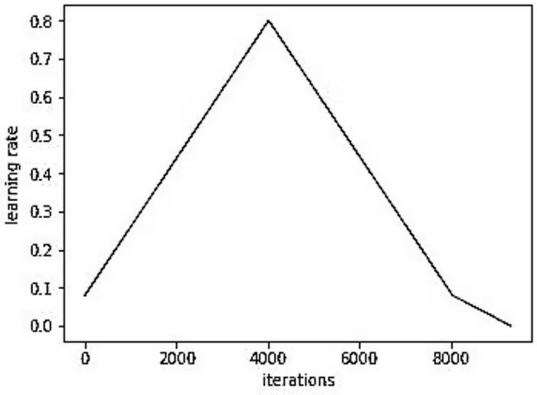
How do I leave the saddle point?

These points, where the gradient is zero but which are not minima, will require a great many iterations; however, by periodically increasing the learning rate, we will be able to escape from these points easily and converge more quickly

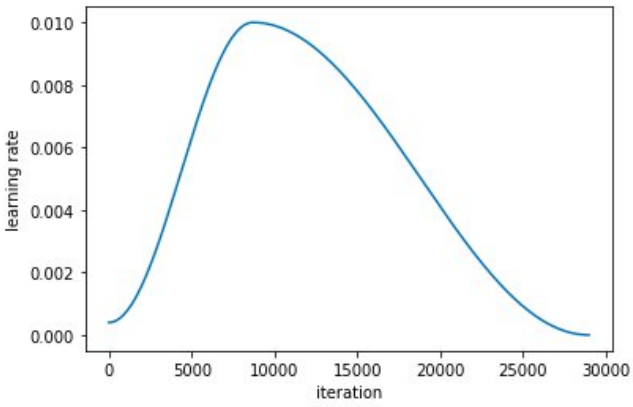
One Cycle Learning Rate

One cycle is enough! [A disciplined approach to neural network hyper-parameters](#) - *Leslie N. Smith* *2018*

Proposition initiale



cosine annealing : Recommandation par FastAI

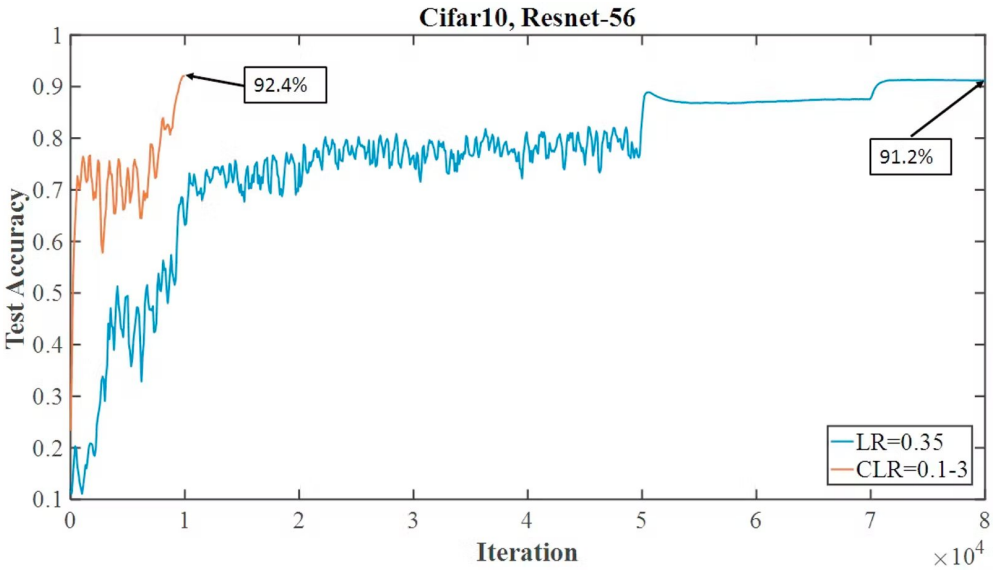


16

In 2018, Leslie N. Smith realised that, rather than running several cycles, it is more efficient to run just one large cycle of increasing and decreasing the learning rate, known as the ‘One Cycle Learning Rate’. This leads to a ‘super-convergence’ effect, whereby the model is forced to converge rapidly with a high learning rate following a phase of gradual increase, followed by a phase of decrease towards a learning rate lower than the initial LR.

The original proposal involved a linear increase and decrease, but FastAI later proposed a more efficient variation using a sinusoidal curve. In both cases, the cycle ends with a significant reduction in the learning rate during the final iterations.

One Cycle Learning Rate - *Super convergence*



Faster convergence for equivalent final precision

In his paper, Leslie N. Smith demonstrates that, using a ResNet 56 network and the CIFAR-10 dataset, convergence is achieved more quickly with a cyclic learning rate than with a constant learning rate, whilst achieving a similar level of accuracy.

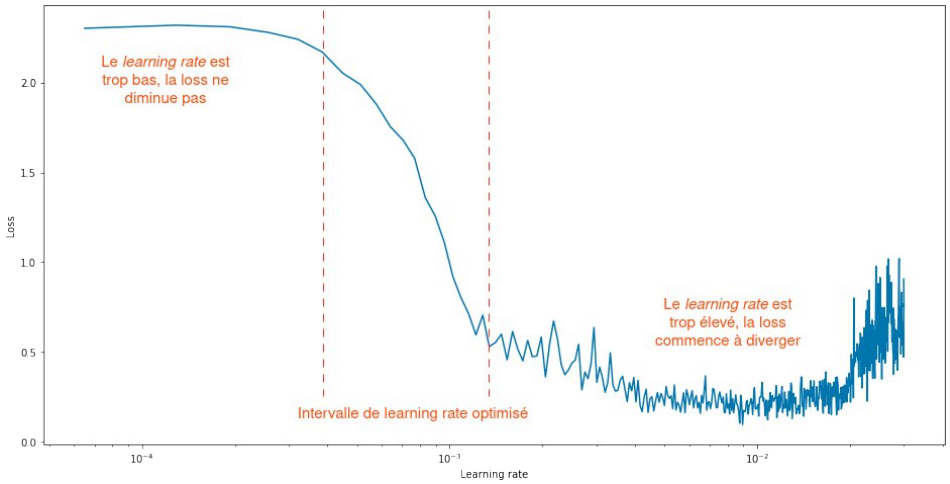
Learning Rate Finder

Goal: Find the **optimal learning rate** values for your model, particularly for **the maximum value** of a *cyclic scheduler*

Run your model over a few epochs by increasing its learning rate

- **Start of loss reduction**
- **→ Minimal learning rate**

- **Start of loss variation**
- **→ Maximum learning rate**



To find the minimum and maximum values of the cyclic learning rate, we use the learning rate finder technique.

We will train a model for a few epochs, starting with a very low learning rate which will gradually increase until it reaches a very high value.

By plotting the loss curve for this short training run, we will be able to observe two things:

- Initially, the loss does not decrease until we reach a learning rate high enough to have a positive effect on the model. This value will be the minimum.

- The loss decreases fairly steadily until it begins to fluctuate significantly and diverge. We can therefore find the maximum learning rate that has a positive effect on the loss without causing it to diverge.

TROUVER SOURCE LR FINDER

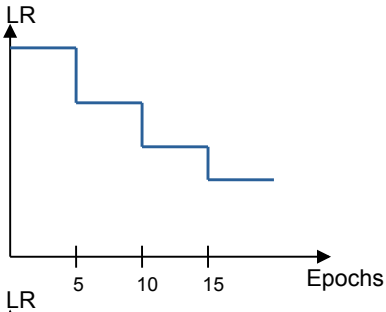
Learning Rate Scheduler

Each **scheduler** has *its own settings*

```
import torch.optim as opt

scheduler = opt.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

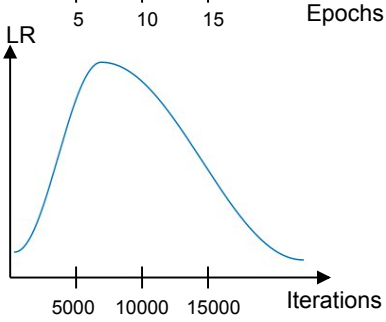
for epoch in range(100):
        train(...)
        validate(...)
        scheduler.step()
```




```
import torch.optim as opt

scheduler = opt.lr_scheduler.CyclicLR(optimizer, base_lr=0.01, max_lr=0.1)


for epoch in range(10):
        for batch in data_loader:
                train_batch(...)
                scheduler.step()
```



All the types of learning rate schedulers we have looked at in this course are implemented in the PyTorch library.

It should be noted that cyclic learning rate schedulers update their values with every new batch, unlike the others, which update their values at the start of each epoch.

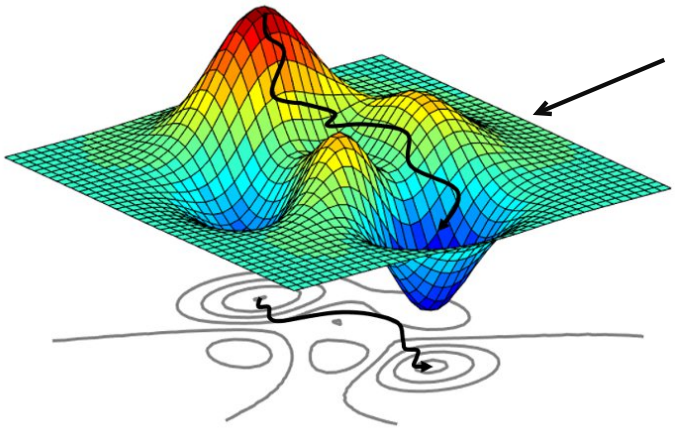
Gradient Descent Optimizer

SGD ◂
ADAM◂
ADAMW ◂

In this chapter, we will examine how the two most commonly used optimisers work and the importance of their parameters for machine learning.

Optimizer - SGD

The **optimizer** is the algorithm that **controls the gradient descent** and **the minimum search** with the aim of optimizing the learning time and the final metric.



***SGD** = **S**tochastic **G**radient **D**escent*
Calculating the Gradient and updating the weights at each batch

+ Batch size and learning rate adaptable according to conflicting needs:
- Exploration to find the best local minimum
- Acceleration of gradient descent

Reminder: the optimiser is the algorithm that controls gradient descent and the search for the minimum in the loss function. Its aim is to make the gradient descent converge to the global minimum (or best local minimum) as quickly as possible.

SGD *with Momentum*

Goal: Take **previous gradients** into consideration for **faster** gradient descent.

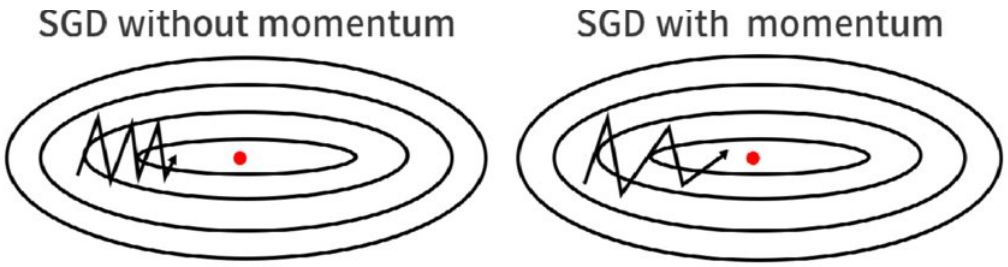
$$m_0 = 0$$

$$m_i = \beta * m_{i-1} + (1 - \beta) * g_i$$

$$\theta_i = \theta_{i-1} - \alpha * m_i$$

Momentum coefficient

+ Allows you to **converge more quickly**
- **No guarantee** that momentum will take us in the right direction

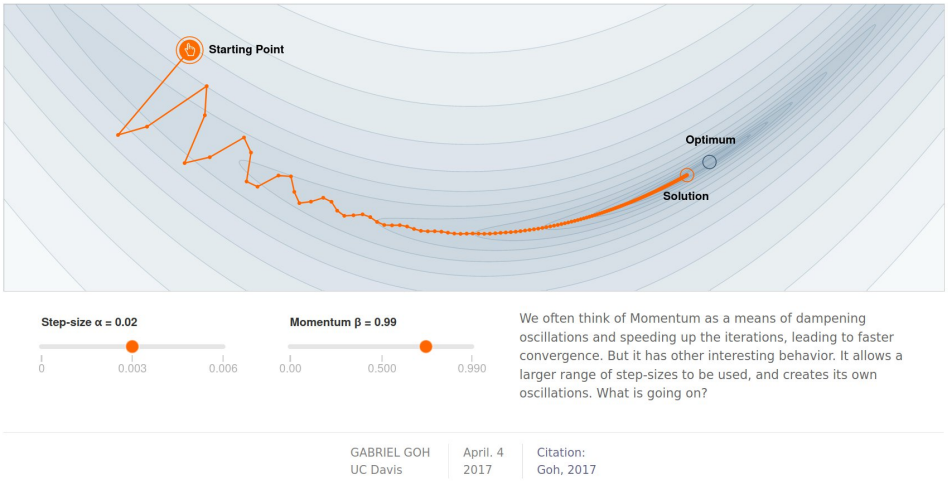


To limit instabilities and converge more quickly, we once came up with the idea of using momentum, a form of ‘optimizer memory’.

To use momentum, optimisers introduce a value m, which stores the previous gradients in memory using a moving average. The significance of this value can vary depending on the momentum coefficient we assign to our optimiser.

Using this gradient memory, when there is a steep gradient, the weight updates will be larger, enabling us to converge more quickly.

Why Momentum Works ?



Step-size α = 0.02

0 0.003 0.006

Momentum β = 0.99

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

GABRIEL GOH
UC Davis

April. 4
2017

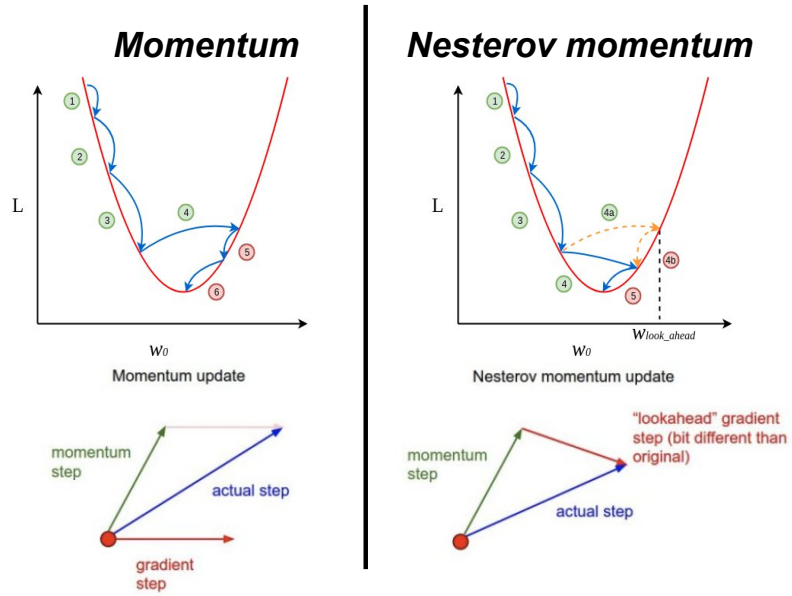
Citation:
Goh, 2017

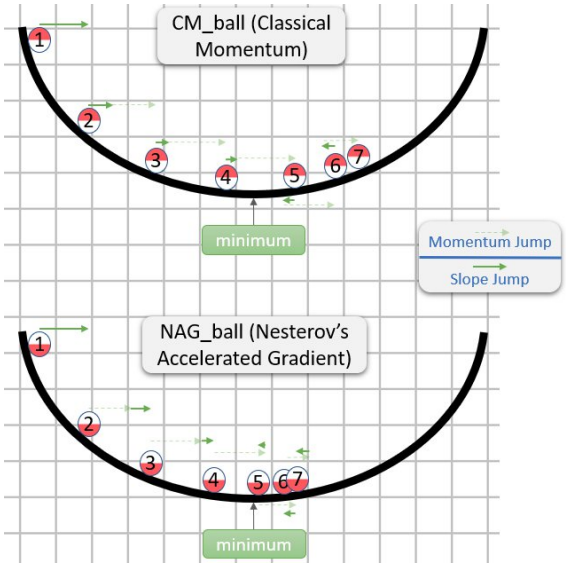
https://distill.pub/2017/momentum/

In some cases where the momentum becomes too strong, the gradient update may be too large and fail to find the minimum loss value we are seeking.

To counter this sort of problem, a version of momentum known as ‘Nesterov’s momentum’ does not calculate the gradient at the current position, but rather the gradient at an intermediate position (lookahead) calculated by applying the current momentum. If the momentum is too high and causes us to overshoot the minimum we wish to reach, the modified gradient should dampen the weight update.

Nesterov accelerated gradient





In Nesterov momentum, we first look ahead in the momentum direction, then compute the gradient at the anticipated position.

In some cases where the momentum becomes too strong, the gradient update may be too large and fail to find the minimum loss value we are seeking.

To counter this sort of problem, a version of momentum known as 'Nesterov's momentum' does not calculate the gradient at the current position, but rather the gradient at an intermediate position (lookahead) calculated by applying the current momentum. If the momentum is too high and causes us to overshoot the minimum we wish to reach, the modified gradient should dampen the weight update.

Root Mean Square Propagation

Each **gradient element is squared** to make large gradients easier to detect

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + (1-\beta)\mathbf{g}_t^2$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\alpha}{\sqrt{m_t} + \epsilon} g_t$$

Normalisation :
→ Large recent squared gradients ⇒ $\boldsymbol{m_t}$ is large ⇒ smaller effective learning rate
→ Small recent squared gradients ⇒ $\boldsymbol{m_t}$ is small ⇒ larger effective learning rate

RMSProp highlights large gradients,
without letting them produce excessively large weight updates

RMSProp (short for Root Mean Square Propagation) is an optimiser that harnesses the momentum of high gradients whilst limiting their ‘overshoot’ effect.

By using a moving average of the squares of the gradients, RMSProp then divides the learning rate by this average to obtain an adaptive learning rate that avoids overly abrupt updates.

REVOIR RMSPROP

ADAptive Moment estimation

$$m_i = \beta_1 * m_{i-1} + (1 - \beta_1) * g_i$$ First moment : SGD like

$$v_i = \beta_2 * v_{i-1} + (1 - \beta_2) * g_i^2$$ Second moment : RMSPROP like

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^i}$$

$$\hat{v}_i = \frac{v_i}{1 - \beta_2^i}$$

Normalisation : Prevents the optimizer from underestimating gradients at the beginning of training

$$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i + \epsilon}} * \hat{m}_i$$ Normalisation : RMSPROP like

Adam modifies both the magnitude and the direction of the update for each weight. 26

The Adam (Adaptive Moment Estimation) optimiser builds on the concept of momentum by maintaining a moving average of the previous gradients (first moment), the weight of which varies depending on the value of the coefficient β1.

Adam adds another value (second moment), which is the moving average of the squares of the previous gradients; the square root of this is then calculated, corresponding to the moving uncentred variance of the previous gradients.

An additional step is taken to reduce bias in the early iterations, where the moving averages are calculated over a small number of values.

To update the weights, we subtract the first moment m divided by the square root of the second moment v from the network's weights, multiplied by the learning rate provided as input.

This optimiser therefore has the advantage of incorporating momentum, but also another parameter that prevents momentum from becoming too dominant in the weight updates. These two mechanisms enable Adam to dampen the oscillatory effects in the gradient descent.

MomentUm Orthogonalized by Newton-Schulz

Algorithm 2 Muon

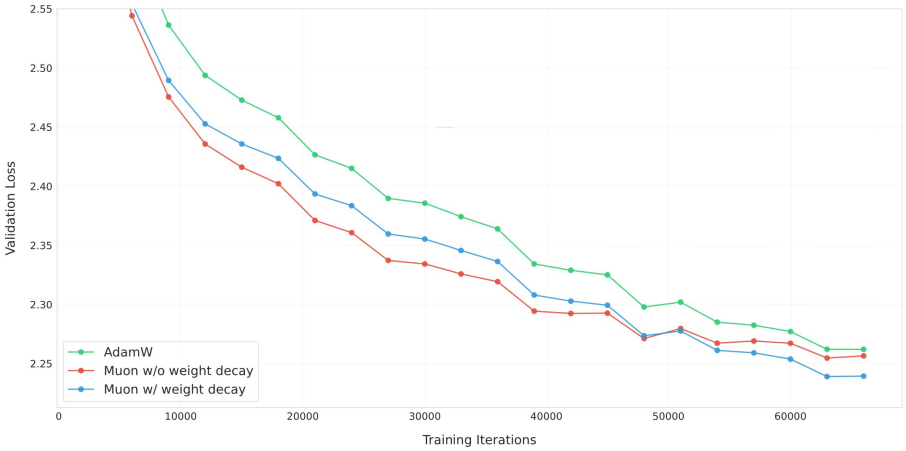
Require: Learning rate η, momentum μ

1: Initialize $B_0 \leftarrow 0$
2: **for** $t = 1, \ldots$ **do**
3: Compute gradient $G_t \leftarrow \nabla_\theta \mathcal{L}_t(\theta_{t-1})$
4: $B_t \leftarrow \mu B_{t-1} + G_t$ Adjust the gradient using momentum
5: $O_t \leftarrow \mathrm{NewtonSchulz5}(B_t)$ Preserving its direction while normalizing its magnitude
6: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
7: **end for**
8: **return** θ_t

Muon emphasizes the orientation of the momentum-adjusted gradient rather than its magnitude

MomentUm Orthogonalized by Newton-Schulz

Muon trains Transformers faster than Adam but is less well suited to CNN architectures

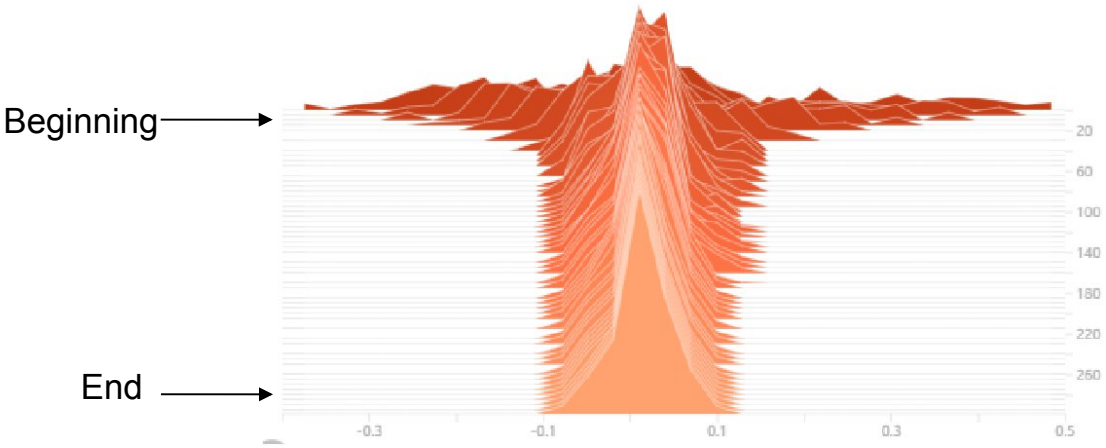


MUON is scalable for llm training, 2025, Jingyuan Liu

In Transformers, Muon can be applied independently to each attention head

Weight decay

Distribution of weights during learning:



During training the weights tend toward zero

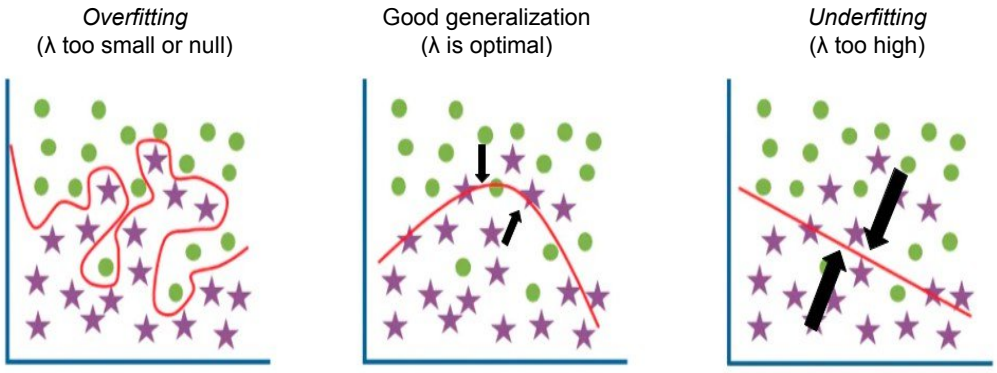
Lors des entraînements de réseaux de neurones, on remarque généralement que les modèles qui généralisent bien ont une distribution des poids de plus en plus proche de zéro.

La raison est que des modèles aux poids trop élevés sont plus à même de faire de l'overfitting des données.

Weight decay

$$\theta_t \leftarrow (1 - \lambda)\,\theta_{t-1} - lr \times g_t$$

λ : *weight decay* parameter *(between 0 and 0.1)*



The weight decay technique force the weights to converge towards values close to zero

Le *weight decay* va garder les poids du modèle proches de zéro. Dans ce but les optimiseurs vont prendre un paramètre supplémentaire λ qui va représenter la proportion de réduction des poids à chaque mise à jour.

Si λ est trop faible ou nul les poids vont moins tendre vers zéro et il est possible que le modèle devienne trop adapté aux données d'entraînement et donne de mauvais résultats avec les données de test (*Overfitting*).

A l'inverse si λ est trop élevé le modèle va commencer à trop généraliser et aura une mauvaise précision.

Lorsqu'il est bien configuré le *weight decay* va réduire le temps d'entraînement en accélérant la vitesse de convergence des poids vers zéro.

Weight decay and decoupled weight decay

Problem : ADAM

For i = 1 to ...

$g_i = \nabla_\theta f_i(\theta_{i-1}) + \lambda\theta_{i-1}$ ⚠

$m_i = \beta_1 * m_{i-1} + (1-\beta_1) * g_i$

$v_i = \beta_2 * v_{i-1} + (1-\beta_2) * g_i^2$

$\hat{m}_i = \frac{m_i}{1-\beta_1^i}$

$\hat{v}_i = \frac{v_i}{1-\beta_2^i}$

$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i + \epsilon}} * \hat{m}_i$

Return θ_i

Weight decay

Weight decay is affected by Adam's momentum and adaptive scaling

Solution : ADAMW

For i = 1 to ...

$g_i = \nabla_\theta f_i(\theta_{i-1})$

$m_i = \beta_1 * m_{i-1} + (1-\beta_1) * g_i$

$v_i = \beta_2 * v_{i-1} + (1-\beta_2) * g_i^2$

$\hat{m}_i = \frac{m_i}{1-\beta_1^i}$

$\hat{v}_i = \frac{v_i}{1-\beta_2^i}$

$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i + \epsilon}} * \hat{m}_i - \alpha\lambda\theta_{i-1}$

Return θ_i

Decoupled weight decay

SGD and SGDW are **roughly equivalent** in performance.
However **AdamW is noticeably better than Adam!!**

Il existe deux implémentations différentes du *weight decay*. Dans la version initiale, on va ajouter au gradient la valeur des poids multipliée par le coefficient de *weight decay.* Cette méthode est utilisée dans les optimiseurs SGD et ADAM.

Avec le *decoupled weight decay* on va plutôt soustraire directement la valeur des poids multipliée par le coefficient de *weight decay* au niveau de la mise à jour des poids.

Optimization of large batches

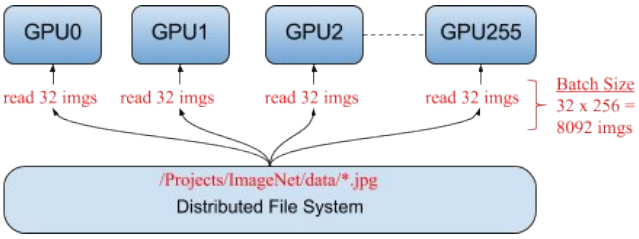
Large batches issues ◂

Learning Rate Scaling & Batch Schedulers ◂

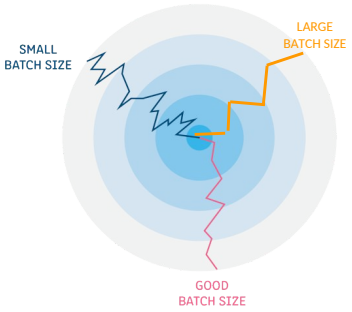
Large batches optimizers ◂

Dans ce chapitre, nous allons nous intéresser aux problématiques que l'on rencontre quand la taille de batch devient trop grande et nous allons étudier les différentes solutions qui existent pour faire face à ces problématiques.

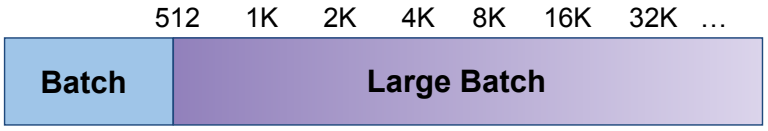
Large Batches with Data Parallelism



Data Parallelism: This parallelism generates very large batches



Problem:
Batch that are too large (> 512) tend to result in **poorer performance**



33

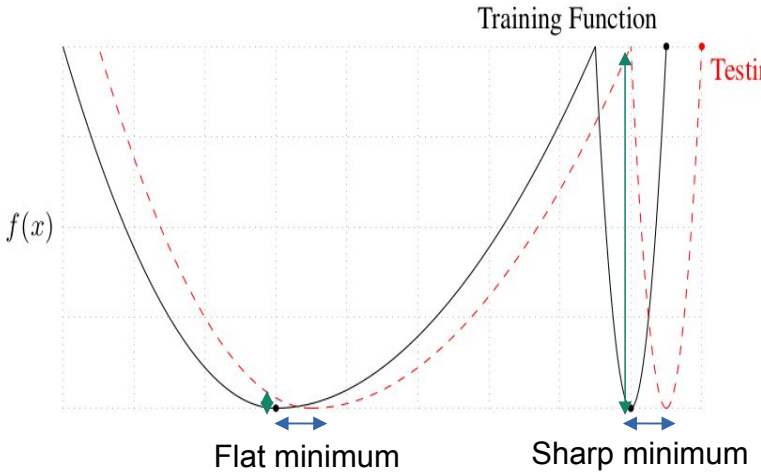
Lorsqu'on utilise le *data parallelism* la taille de *batch* globale va devenir la taille de *batch* initiale multipliée par le nombre de GPU, cette taille globale peut devenir rapidement importante.

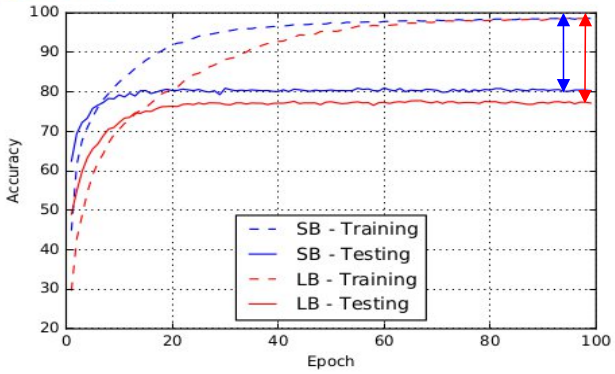
Le problème étant que lorsque la taille de batch devient trop grande, on observe que les modèles ont tendance à moins bien généraliser.

Large Batches

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, Ping Tak Peter Tang





Comparison of training a convolutional network with small batch (SB) and large batch (LB) on CIFAR 10

The *larger the batch*, the more the model tends to converge towards **Sharp minima**.

Dans le papier *On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima*, les auteurs expliquent que ce problème de mauvaise généralisation est causé par le fait que plus la taille de *batch* est grande plus le modèle va avoir tendance à converger vers des minimums pentus (*sharp minimum*) et étroits au lieu de minimums larges et plats (*flat minimum*).

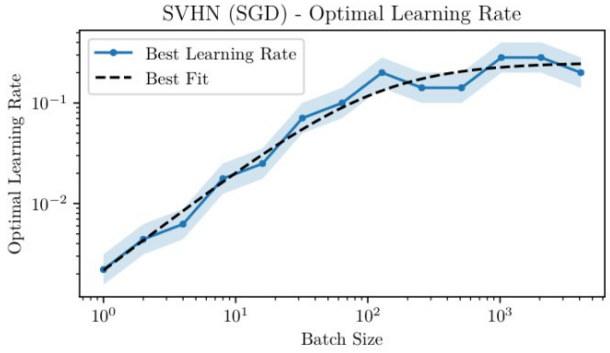
Les *sharp minima* vont moins bien généraliser, car contrairement aux *flat minimuma* une petite différence entre les données d'entraînement et les données de test vont amener une grosse différence sur la sortie du modèle.

Large Batches : Learning rate scaling

When the *size of the global* batch **is considerably increased**, it is often *necessary* to **scale the learning rate :**

Relationship Between Learning Rate and Batch Size : ADAM $\alpha \rightarrow \sqrt{N} * \alpha$

SGD $\alpha \rightarrow N * \alpha$



An Empirical Model of Large-Batch Training
Sam McCandlish, Jared Kaplan, Dario Amodei

Beyond a certain point, training reaches a plateau, and larger batches provide little additional benefit.

Lorsqu'on a déjà un *learning rate* adapté à notre modèle, mais que l'on souhaite mettre notre apprentissage à l'échelle sur plus de GPU (ce qui peut considérablement augmenter notre taille te *batch)* il est nécessaire de faire une mise à l'échelle de notre *learning rate*.

Pour cela, deux approches sont possibles : une croissance linéaire (multiplication par le nombre de processus) ou une croissance en racine carrée (multiplication par la racine carrée du nombre de processus).

Dans le but de tester empiriquement laquelle de ces deux approches est préférable, des chercheurs de OpenAI ont tracé une courbe des *learning rate* optimaux en fonction du nombre de GPU avec SGD. On peut voir que pour un petit nombre de GPU la valeur augmente linéairement, mais au bout d'un certain nombre de GPU on est plus proche d'un croissance en racine carrée.

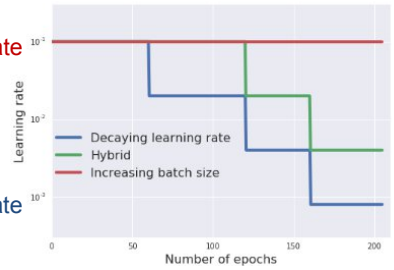
Batch Size Scheduler

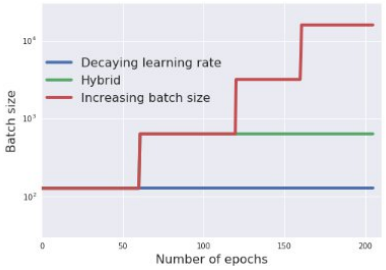
=> Alternative to Learning Rate Scheduler

Don’t decay the learning rate, increase the batch size

High Learning Rate

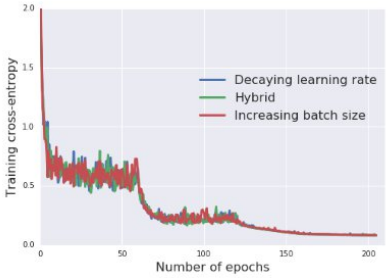
Low Learning Rate

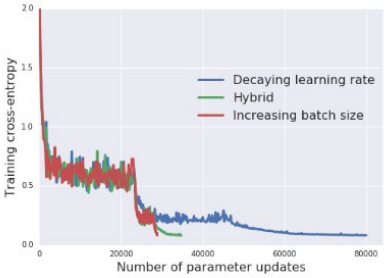




Large Batch

Small Batch

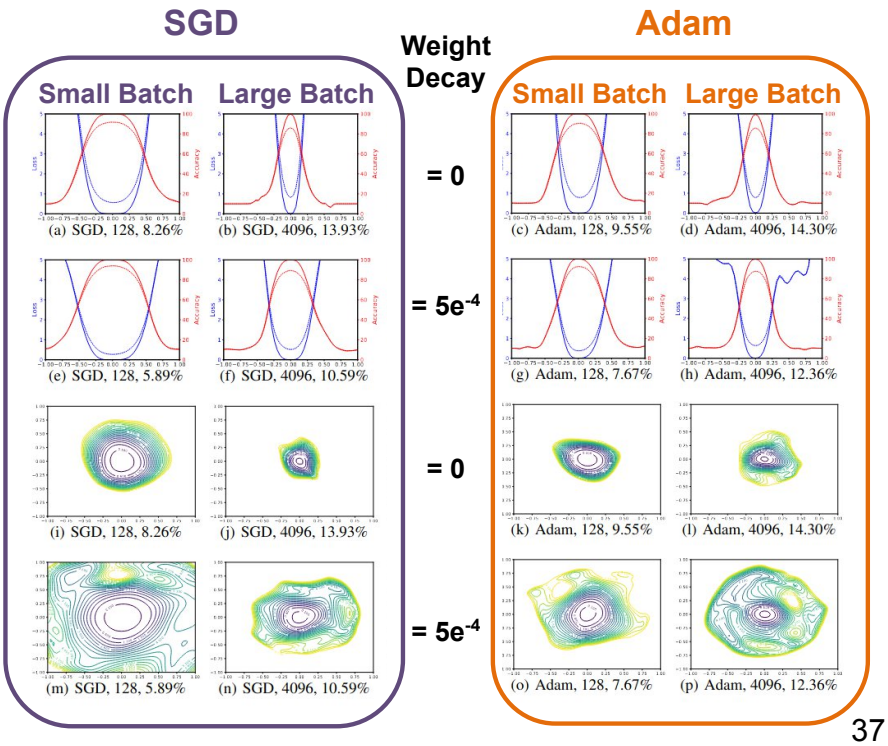




Une technique alternative au *learning rate scheduler* est d’augmenter dynamiquement la taille du *batch* pendant l’apprentissage. Cette technique, qui permet d’atteindre les même performances qu’avec un *learning rate scheduler*, aurait l’avantage de faire moins de mises à jour des paramètres, ce qui permet d’entraîner son modèle plus rapidement.

Large Batches

Small Batch	Large Batch
Flat Minimum	Sharp Minimum
- Test Loss	+ Test Loss
Slow Descent	Fast Descent



En observant la visualisation 2D de la *loss* de différents apprentissages, on peut voir les effets des optimiseurs et paramètres utilisés lors de ces apprentissages.

On observe que des petites tailles de batch ont tendance à faire des *flat minimum* ce qui peut causer un apprentissage plus lent en contrepartie d'une petite différence entre l'apprentissage et les tests.

A l'inverse, une grande taille de batch va amener plus de *sharp minimum* qui vont amener une plus grosse différence entre les *loss* d'apprentissage et de test mais un apprentissage plus rapide.

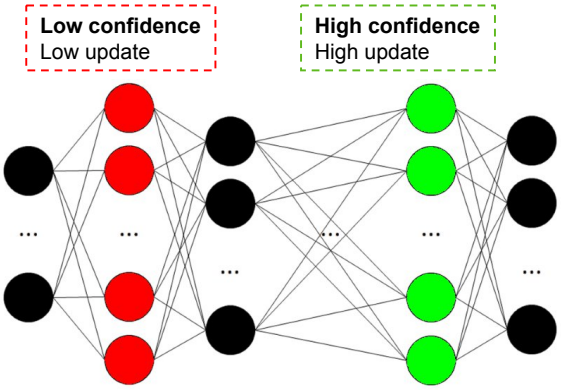
L'optimiseur SGD tend vers des *flat minima* tandis que ADAM tend plutôt vers des *sharp minima.*

L'ajout d'un *weight decay* corrige l'effet des différentes tendances et réduit les chances de tomber dans un *sharp minimum.*

Layer-wise Adaptive Rate Scaling

Adapt the effective learning rate of each layer using a trust ratio based on the relative norms of its weights and gradients.

$$\lambda^{(l)} = \alpha_{\text{trust}} \frac{\left\|w^{(l)}\right\|}{\left\|g^{(l)}\right\| + \beta \left\|w^{(l)}\right\|} \approx \alpha_{\text{trust}} \frac{\text{size of weights}}{\text{size of proposed update}}$$



L'optimiseur LARS (*Layer-wise Adaptative Rate Scaling*) est assez similaire à SGD, mais va rajouter une variable R appelé le ratio de confiance. Ce ratio va être recalculé à chaque mise à jour du modèle pour chaque couche de neurone.

La variable R1 représente la norme des poids de la couche et R2 représente la norme des changements des poids de la couche avec le *weight decay.*

En multipliant le *learning rate* par le ratio R1/R2 on va pouvoir adapter la vitesse de modification des poids de chaque couche en fonction de la confiance qu'on leur donne.

Optimizer implementation

SGD	`import torch.optim as opt` SGD_optimizer = opt.SGD(params, lr, momentum=0, weight_decay=0, nesterov=False, ...)
ADAMW	`import torch.optim as opt` ADAM_optimizer = opt.AdamW(params, lr=0.001, betas=(0.9, 0.999), weight_decay=0.05,..)
LAMB (LARS with adam)	`from apex.optimizers import FusedLamb` LAMB_optimizer = FusedLamb(params, lr=0.001, betas=(0.9, 0.999), weight_decay=0, adam_w_mode=True)
Muon	`import torch.optim as opt` MUON_optimizer = opt.Muon(params, lr=0.02, Momentum=0.95, weight_decay=0.05, ...)

Are large-batch optimizers still useful?

LARS enables very large-batch CNN training, but the original paper does **not prove it is necessary**.
Source: Large Batch Training of Convolutional Networks

LARS may be better tuned rather than inherently better, **standard optimizers can outperform it.**
Source: A Large Batch Optimizer Reality Check

Muon is becoming a standard choice for training frontier LLMs, especially with large batches.
Source: Muon is Scalable for LLM Training

Quand on fait du *Data Parallelism*, gérer des gros *batches* est inévitable. Nous allons donc devoir affronter ce problème de *sharp minima*.

Il existe différents outils pour faire face à cette problématique.

En plus du *weight decay* et des optimiseurs adaptatifs que nous avons vus, il est recommandé d'utiliser des *learning rate schedulers* ainsi que des optimiseurs adaptés au *large batches* comme LARS et LAMB que nous allons voir plus tard.

BLOOM example

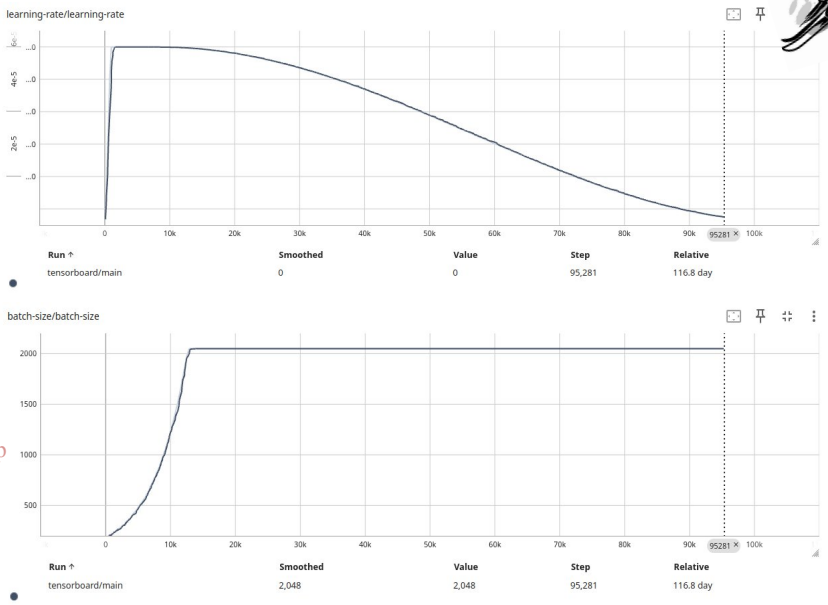
95281 steps (116.8 days)

AdamW,
β1=0.9, β2=0.95, eps=1e−8

Weight Decay of 0.1

LR Scheduler
- peak=6e-5
- warmup over 375M tokens
- cosine decay for learning rate down to 10% of its value, over 410B tokens

Batch Scheduler
- start from 32k tokens (GBS=16)
- increase linearly to 4.2M tokens/step (GBS=2048) over ~20B tokens
- then continue at 4.2M tokens/step (GBS=2048) for 430B tokens



Quand on fait du *Data Parallelism*, gérer des gros *batches* est inévitable. Nous allons donc devoir affronter ce problème de *sharp minima*.

Il existe différents outils pour faire face à cette problématique.

En plus du *weight decay* et des optimiseurs adaptatifs que nous avons vus, il est recommandé d'utiliser des *learning rate schedulers* ainsi que des optimiseurs adaptés au *large batches* comme LARS et LAMB que nous allons voir plus tard.

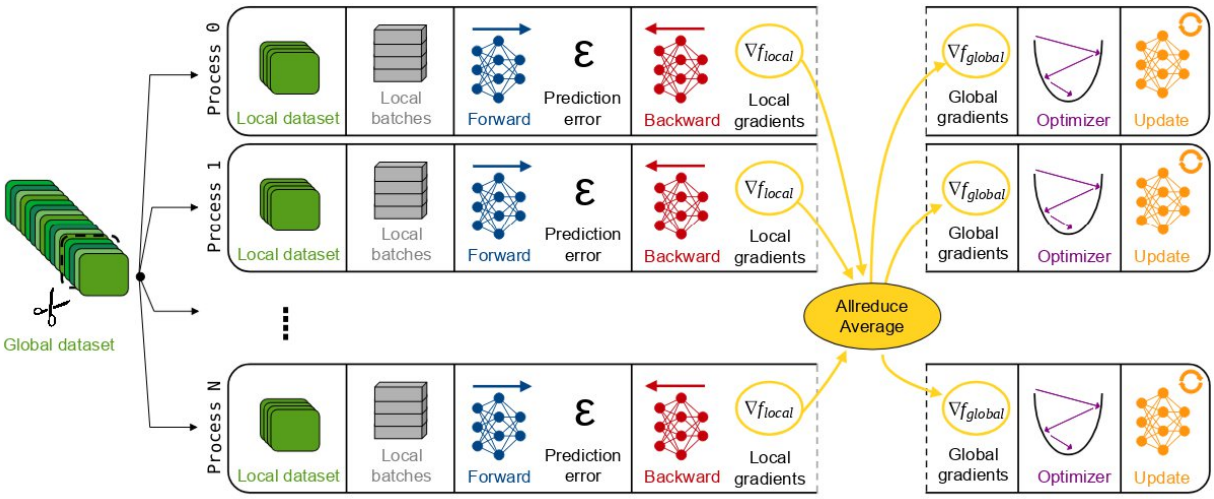
Reducing Optimizer Communication Costs

Communication optimization◂

PowerSGD ◂

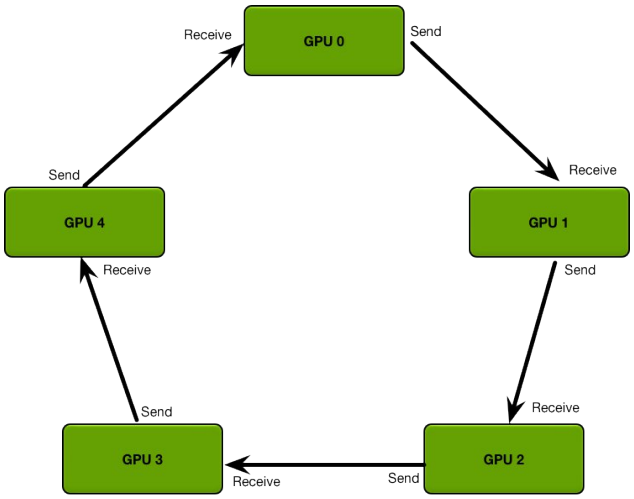
DiLoCo ◂

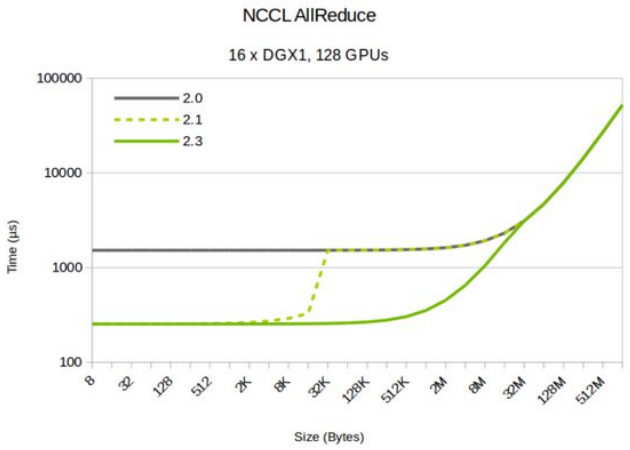
Distributed Data Parallel bottleneck



Communication optimization

Ring all reduce



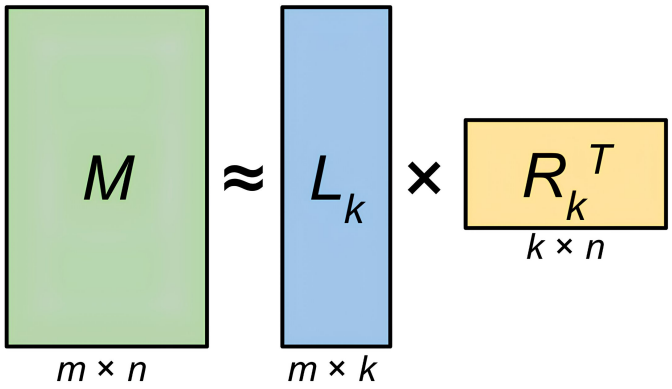


The NCCL version can affect performance

Each GPU sends a fraction of the gradient to the next GPU over N−1 communication steps.

Gradient compression - PowerSGD

The idea is to decompose the weight matrix into two smaller matrices



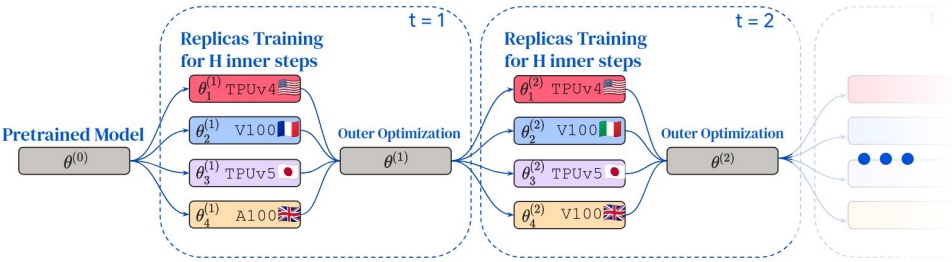
This reduces communication by transmitting two smaller matrices.

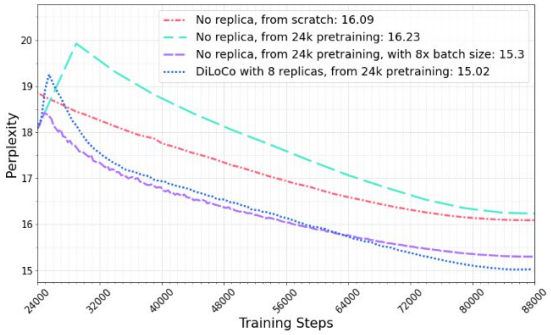
Asynchronous Optimization - DiLoCo

Algorithm 1 DiLoCo Algorithm

Require: Initial model $\theta^{(0)}$
Require: k workers
Require: Data shards $\{\mathcal{D}_1, \ldots, \mathcal{D}_k\}$
Require: Optimizers `InnerOpt` and `OuterOpt`

for `outer step` $t = 1 \ldots T$ **do**
  for `worker` $i = 1 \ldots k$ **do**
    $\theta_i^{(t)} \leftarrow \theta^{(t-1)}$
    for `inner step` $h = 1 \ldots H$ **do**
      $x \sim \mathcal{D}_i$
      $\mathcal{L} \leftarrow f(x, \theta_i^{(t)})$
      ▷ Inner optimization:
      $\theta_i^{(t)} \leftarrow \texttt{InnerOpt}(\theta_i^{(t)}, \nabla_{\mathcal{L}})$
    end for
  end for
  ▷ Averaging outer gradients:
  $\Delta^{(t)} \leftarrow \frac{1}{k} \sum_{i=1}^{k} (\theta^{(t-1)} - \theta_i^{(t)})$
  ▷ Outer optimization:
  $\theta^{(t)} \leftarrow \texttt{OuterOpt}(\theta^{(t-1)}, \Delta^{(t)})$
end for





LLM training may not always require globally synchronous gradients at every step

Pratice : Learning rate + Optimiseurs





Goals :

- Edit the **learning rate scheduler**
- Edit the **optimizer**
- Do training with **large batches**

→ From **JupyterHub**:

- ➢ Launch an interactive instance
- ➢ Go to the tp_optimizers folder
- ➢ Open the DLO-JZ_Optimizers notebook